

PRESENTATION COMPANION

Chronos Presentation Guide

Pitch, live-demo narration, likely questions, and honest technical answers

Python proof of concept | Teacher presentation edition | August 2026

One-line idea: Chronos is a Git-like versioned data store that uses content hashes to share unchanged tree nodes between versions and skip unchanged subtrees during comparison.

20-second pitch

Normal snapshots copy an entire database state, while operation logs only remember the commands. Chronos keeps version roots in a hash-based tree. If data does not change, its node hash does not change, so the same node is reused. When two versions are compared, equal hashes let us skip whole subtrees. In our CSV dry run, version 2 reused 1,405 nodes and created only 15 new ones, giving 98.9% structural sharing.

The three facts to land

- The root hash is a compact identity for one complete version.
- Matching subtree hashes prove that the underlying content is identical, so diff stops there.
- The prototype proves structural sharing and hash-pruned diff; durability and incremental commits are future work.

Presentation rule: Do not claim that Chronos is faster at everything. Say exactly where it helps, show the evidence, and state the prototype boundary before your teacher has to find it.

1. Your one-minute pitch

Databases usually preserve history in one of two ways. A state-based system stores complete snapshots, which makes an old version easy to read but duplicates unchanged data. An operational system stores the commands that changed the data, which is compact and fast for short histories but may need replay and depends on complete command history. Chronos combines useful ideas from both. It represents each state as a content-addressed hash tree and can optionally retain SQL history as metadata. A commit stores a root hash. Unchanged subtrees keep the same hashes and are shared rather than copied. During diff, Chronos compares the two roots and descends only where hashes differ. Our current Python prototype demonstrates this behavior with CSV exports from before and after MySQL changes.

2. The mental model

Term	Simple meaning	Why it matters
Node hash	A fingerprint of a node's content	Equal content produces the same identifier
Root hash	The fingerprint representing a complete version	A small reference identifies the full state
Commit	A version number mapped to one root hash	Creates immutable history inside the session
Structural sharing	Two versions reference the same unchanged nodes	Avoids duplicating identical subtrees
Diff pruning	Stop when two compared hashes match	Avoids scanning keys below unchanged subtrees

3. Architecture in one view

Input	Chronos processing	Result
Before CSV	Canonical table:primary-key records	Commit v1 and root hash
SQL text	Attached as audit metadata; not executed	Human-readable command history
After CSV	Hash tree commit plus parallel root diff	Commit v2, reuse stats, changed rows

Core sentence: The SQL file tells us what was intended; the before-and-after CSV states tell us what actually changed.

4. Live demo script

```
python csv_snapshot_demo.py data/users_before.csv data/users_after.csv --table users --primary-key id --sql data/changes.sql
```

1. Before running, say: 'I have a before export, the SQL history, and an after export from the same table.'
2. At commit v1, point to the first root and explain that the initial version must create every required node.
3. At commit v2, point to the different root, 15 new nodes, 1,405 reused nodes, and 98.9% sharing.
4. At diff, show the four affected records: two updates, one insertion, and one deletion.
5. Finish with the traversal evidence: only 81 node pairs were compared and 65 matching subtrees were skipped.

Numbers from the verified CSV dry run

Evidence	Observed result	What to say
Commit v1	1,421 new nodes	This constructs the first complete tree.
Commit v2	15 new / 1,405 reused	Only changed paths receive different content hashes.
Sharing	98.9%	Most of the second version points to existing nodes.
Changed records	4	Two updates, one add, and one delete were found.
Diff work	81 compared / 65 skipped	Equal hashes prevented deeper traversal.

How to narrate the changed rows

- users:10 - status changed from active to inactive.
- users:500 - balance changed from 150.00 to 999.00.
- users:900 - the row was deleted.
- users:1001 - a new row was inserted.

If asked why the root changed: A changed leaf receives a new hash. That changes each ancestor on its path, including the root, while unrelated branches keep their old hashes.

5. Ten likely questions and strong answers

Q1. What problem are you solving?

Short answer: Efficient version history: preserve complete version identities without copying identical subtrees.

Full snapshots repeatedly store unchanged data, while operation logs describe change history rather than directly representing every state. Chronos explores a third design: every version has a root hash, but unchanged content is automatically shared. The prototype focuses on proving that versions can share most of their structure and that comparison can avoid scanning unchanged areas.

Q2. How does `commit()` work?

Short answer: It hashes records from the leaves upward and stores the final root as the version identity.

The CSV loader converts each row into a canonical key such as `users:500`. Chronos places those records into a fixed-depth tree. A leaf hash depends on its sorted records, and an internal-node hash depends on its sorted child hashes. The final root hash is stored against a version number. During the second commit, any node whose content hash already exists is reused rather than inserted again.

Q3. What exactly is diff?

Short answer: It finds added, deleted, or updated keys while skipping branches whose hashes already prove equality.

Diff means finding which keys have different values between two committed versions. Chronos compares the two root nodes. If their hashes match, the versions are identical. If they differ, it compares corresponding children. Whenever a pair of child hashes matches, that entire branch is skipped. Only different branches are followed until the changed records are found.

Q4. How do you know nodes are really shared rather than copied?

Short answer: Identical content maps to the same hash and the same node-store entry.

Nodes are stored in a dictionary keyed by their SHA-256 content hash. Inserting identical content produces the same hash and therefore references the existing dictionary entry. The commit statistics count whether each constructed node hash was already present. In the verified second CSV commit, 1,405 encountered nodes already existed and only 15 were new.

Questions 5-7: model comparison and command history

Q5. Is Chronos state-based, operational, or both?

Short answer: State-based for truth and version identity; operational history can be attached for auditability.

Its version identity is state-based because each root represents the resulting data state. It can also retain operational history, such as SQL text, as audit metadata. That is why the project is presented as a hybrid direction. However, the current diff is calculated from the committed states, not by replaying the SQL commands.

Q6. Why is the operational model faster than Chronos in your benchmark?

Short answer: The workload favors a short log, and our prototype commit still rebuilds the tree; the result is honest, not a failure.

The benchmark compares adjacent versions separated by only a small number of operations. An operation log can simply read those few log entries, so it is expected to win. Chronos also currently rebuilds the candidate tree from the complete input state during commit, making its commit time much slower. Chronos is most valuable when we need state integrity, content sharing, or diffs across histories where replaying a long operation log becomes less attractive. The next engineering step is incremental path updates.

Q7. Does Chronos execute or permanently store the SQL commands?

Short answer: The SQL is audit metadata only; MySQL executes it, and the current Chronos session is in memory.

No. In the CSV demonstration, Python reads the SQL file and attaches its text to in-memory commit metadata. It does not parse or execute the SQL. The text file itself remains on disk, but the metadata disappears when the Python process exits. Durable commit metadata belongs in the future persistent version store.

Questions 8-10: integration, correctness, and roadmap

Q8. How would this integrate with MySQL in a real system?

Short answer: Use a transactional MySQL outbox, then let a Chronos adapter produce one version commit per processed transaction or batch.

The practical design is an outbox pattern. A MySQL transaction updates normal tables and inserts a `chronos_outbox` event atomically. A Chronos adapter reads unprocessed events, converts affected rows into canonical key-value records, applies put or delete operations, and publishes one Chronos commit. The CSV workflow is a demonstration adapter; the outbox avoids requiring manual exports and prevents command history from becoming disconnected from committed database changes.

Q9. What about hash collisions or inconsistent serialization?

Short answer: SHA-256 makes accidental collisions impractical, while canonical serialization ensures equal logical content hashes equally.

SHA-256 collisions are theoretically possible but negligibly likely for this prototype. A more immediate correctness risk is inconsistent serialization: the same logical row must always be ordered and encoded the same way before hashing. The implementation therefore uses sorted, canonical content. A production system should use a stable versioned encoding, verify hashes while reading persisted nodes, and define how data types, nulls, and character encodings are normalized.

Q10. What is incomplete, and what would you build next?

Short answer: Incremental path updates first, then commit history and checkout, then durability and recovery.

The current system is an in-memory proof of concept. It does not yet provide durable node storage, a write-ahead log, crash recovery, transactions, checkout, branching, or merge. It also rebuilds the candidate tree from the complete CSV. The highest-priority next feature is put/delete path copying, where one row change rebuilds only its leaf and ancestors. After that come commit objects, checkout, durable content-addressed storage, and recovery.

6. Presentation strategy

Recommended order

1. Start with the problem: snapshots duplicate data and logs represent commands rather than direct states.
2. Explain the one rule: if two subtree hashes match, stop comparing that branch.
3. Run the two-commit CSV demo and point to root, reuse, and diff statistics.
4. Compare the three benchmark models without claiming one universal winner.
5. State the limitations and finish with incremental updates as the next milestone.

What not to overclaim

Avoid saying	Say this instead
Chronos is faster than operational logs.	The best model depends on the workload; short adjacent histories favor logs.
Only changed data is processed during commit.	Today the tree is rebuilt, but existing hashes are reused; incremental updates are next.
The SQL commands are stored permanently.	SQL text is attached in memory and remains in the source file for this demo.
This is already a database replacement.	This is a versioning-engine proof of concept designed for future database integration.
Matching hashes mathematically eliminate every risk.	SHA-256 equality is treated as practical proof, supported by canonical serialization.

If the live demo fails

- Check that the terminal is inside C:/Users/admin/Desktop/Chronos.
- Run `python -m unittest -v` to show the 11 passing correctness tests.
- Use `python demo.py` as the smaller backup demonstration.
- Keep a screenshot of the verified output, but explain the numbers rather than only showing it.

Terms worth remembering

Term	One-line definition
Content addressing	The content hash is used as the node identifier.
Merkle/hash tree	A tree whose parent hashes depend on child hashes.

Term	One-line definition
Immutable node	Once created, the node's content does not change.
Structural sharing	Multiple versions reference identical nodes.
Canonical encoding	A deterministic representation used before hashing.

Final checklist

- Open VS Code in the correct folder and enlarge the terminal text.
- Have the before CSV, after CSV, and changes.sql ready to show.
- Run the tests once before presenting.
- Know the verified numbers: 15 new, 1,405 reused, 98.9% shared, four changed records.
- End by asking whether your teacher would like to see the benchmark or the code for the early-stop rule.

Closing line: Chronos does not claim that history is free; it demonstrates that unchanged data does not need to be copied or re-examined when content hashes already prove it is the same.